

# Python Tools for Evaluating iEoM Simulation Data

---

## User Guide

Joshua Althüser

2026

### 1 Purpose

This small Python package is intended as a post-processing layer for raw simulation data generated with the iEoM library. The iEoM library computes Green's functions by transforming the equations of motion into matrix and Lanczos problems, and the Python package evaluates and filters the resulting data for subsequent plotting.

### 2 Prerequisites and quick setup

The following external packages are required:

- Python (tested with 3.9.25)
  - `setuptools` (tested with version 53.0.0)
  - `wheel` (tested with version 0.36.2)
  - `pybind11` (tested with version 2.13.6)
  - `numpy` (tested with version 2.0.0)
  - `pandas` (tested with version 2.2.2)
- A C++17-capable compiler for building `mrock.cpp_continued_fraction`.
- Boost headers (tested with version 1.78.0).
  - Set the environment variable `BOOST_INCLUDE_DIR` if Boost is installed in a non-standard include path.

If all these are fulfilled, enter the `python` directory and run `pip install -e .`

This will install the module in editable mode, i.e., not shift it out of this project tree.

### 3 Relation to the iEoM output

In the iEoM workflow, the C++ library computes resolvent data by applying Lanczos iterations to suitable starting states. The resulting continued-fraction coefficients are stored as arrays named `a_i` and `b_i` inside resolvent data containers. The Python class `ContinuedFraction` expects a pandas-like data object with a `continuum_boundaries` entry and entries of the form `resolvents.<name>`. If the data were generated by the iEoM library and loaded with `DataLoader.load_panda(...)`, the resulting object can be passed directly to `ContinuedFraction`.

## 4 Package structure

The module `get_data.py` provides a `DataLoader` class to locate and load compressed JSON and pickle files from a configurable data directory. By default, the data directory is resolved relative to the package, but it can also be overridden through the `MROCK_DATA_DIR` environment variable or by passing a path to `DataLoader`. If you downloaded the entire file tree from the repository, you should create a `data` directory in the superproject root dir<sup>1</sup>.

Compressed JSON files are read with `gzip`, decoded with `json`, converted to NumPy arrays when requested, and returned as pandas objects. The same module also contains helper functions such as `hubbard_params` and `continuum_params`, which create parameter dictionaries for constructing data paths. The paths generated by these functions match the default output structure of the respective application.

The module `continued_fraction.py` contains the high-level class `ContinuedFraction`. This class wraps the stored continued-fraction coefficients, determines a termination depth, and delegates numerical evaluation to the compiled backend `cpp_continued_fraction`. It can evaluate continued fractions, spectral densities, real parts, denominators, terminator contributions, and approximate bound-state classifications. It also provides convenience plotting support through `mark_continuum`, which shades the continuum interval in a Matplotlib axis.

The file `cpp_continued_fraction.cpp` implements the performance-critical routines in C++ and exposes them to Python through pybind11. The backend validates one-dimensional input arrays, computes the square-root terminator, evaluates continued fractions<sup>2</sup>, and searches for bound states below the continuum edge. The bound-state classification is designed for finite-energy simple poles and is less reliable for Goldstone-like double-pole structures near  $\omega = 0$ .

The file `FullDiagPurger.py` provides the class `FullDiagPurger`, which post-processes full-diagonalization data. It filters likely artifacts, separates amplitude and phase modes, and stores weaker companion peaks as glimmering modes. It also includes plotting helpers for phase and amplitude eigenvectors, as well as a method for exporting selected data to a dictionary.

Example plotting scripts are provided in the `tests` directories of the applications `Hubbard`, `ContinuumSystem`, and `LatticeCUT`.

## 5 Loading data

A typical analysis starts by creating a `DataLoader`. The loader constructs paths of the form `data_dir/model/subdir/key=value/key=value/.../file`. This convention is implemented by the internal method `_to_path`. For example, the Hubbard example loads `resolvents.json.gz` from model-specific folders using the helper `hubbard_params`.

```
1 from mrock.get_data import DataLoader, hubbard_params
2
3 loader = DataLoader()
4
5 data = loader.load_panda(
6     model="hubbard",
7     subdir="cube/test",
8     file="resolvents.json.gz",
9     **hubbard_params(T=0.0, U=-2.5, V=-0.1)
10 )
```

---

<sup>1</sup>This can, and depending on your infrastructure, probably should, be a soft link to wherever you want to save your data

<sup>2</sup>see, e.g., D. Pettifor and D. Weaire, *The Recursion Method and Its Applications*

## 6 Evaluating continued fractions

After loading data, one creates a `ContinuedFraction` object. During initialization, the object reads the two continuum boundaries, checks that they are ordered, stores their squares, and computes asymptotic coefficients from them. The asymptotic values used internally are

$$a_{\infty} = \frac{\omega_{\text{upper}}^2 + \omega_{\text{lower}}^2}{2}, \quad b_{\infty} = \frac{\omega_{\text{upper}}^2 - \omega_{\text{lower}}^2}{4}.$$

These expressions follow the implementation of `a_infinity` and `b_infinity`. Note that the frequencies are squared because the continued fraction (and the Green's functions in general) are evaluated as functions of  $\omega^2$ .

```
1 import numpy as np
2 import mrock.continued_fraction as cf
3
4 resolvents = cf.ContinuedFraction(data)
5
6 omega = np.linspace(0.0, data["continuum_boundaries"][1] + 0.3, 2000)
7 omega = omega.astype(complex) + 1e-6j
8
9 A_phase = resolvents.spectral_density(omega, "phase_SC")
10 A_higgs = resolvents.spectral_density(omega, "amplitude_SC")
```

The method `continued_fraction` converts scalar or array input to complex NumPy arrays and forwards the evaluation to the C++ backend. The spectral density is computed from the imaginary part of the continued fraction using the normalization factor

$$\mathcal{A}(\omega) = -\frac{1}{\pi} \text{Im } G(\omega).$$

The method `real_part` returns the real part of the same continued fraction. The method `denominator` evaluates the normalized denominator through the C++ backend.

## 7 Termination and square-root terminator

The continued fraction must be truncated at a finite depth. The Python wrapper chooses a termination index by comparing the available coefficients with their asymptotic values and selecting the index with the smallest relative deviation after skipping the first `ignore_first` coefficients. The parameters `ignore_first` and `ignore_last` control the search range for the termination depth. There is no universal choice for these parameters, because terminating too early may miss physical features while going too deep may amplify numerical errors.

When `with_terminator=True`, the backend attaches a square-root terminator determined by the continuum boundaries and the asymptotic coefficients. The C++ implementation uses only the real part of the frequency when computing the terminator. The terminator is complex within the continuum and real outside it.

## 8 Bound states

The method `classify_bound_states` delegates the search for bound states to the C++ backend. The backend scans the interval below the lower continuum edge and searches for zeros of a real-valued denominator. The result is a list of pairs  $(\omega_n, Z_n)$ , where the first entry is an approximate energy and the second entry is an approximate spectral weight. The implementation assumes that finite-energy bound states are simple poles. Note that Goldstone modes or double-pole-like structures are not robust. In those cases, the weight should be computed using the Kramers-Kronig relations and a fit to the real part of the Green's function.

```

1 states = resolvents.classify_bound_states(
2     "amplitude_SC",
3     n_scan=1000,
4     weight_domega=1e-8
5 )
6
7 for energy, weight in states:
8     print(energy, weight)

```

An optional predicate `is_phase_peak` can be passed to remove selected phase peaks from the returned list.

## 9 Post-processing full diagonalization data

The class `FullDiagPurger` is intended for full-diagonalization output rather than Lanczos-only continued-fraction output. It expects amplitude eigenvalues, amplitude weights, amplitude eigenvectors, phase eigenvalues, phase weights, phase eigenvectors, and continuum boundaries in the input mapping.

For each doublet, the mode with the larger weight is assigned to its main channel, and the weaker companion is stored as a glimmering mode. This reflects the fact that, without exact particle-hole symmetry, a physical mode can have a smaller nonzero weight in the “wrong” channel.